Financial Market Data Pipeline

A Complete Data Engineering Portfolio Project

30	15K	9	2yr
Stocks Tracked	Rows Processed	Technologies	Historical Data

Data Engineering Stack

Docker & Docker Compose
Apache Airflow 2.9
Apache Spark 3.5 / PySpark
Medallion Architecture

Cloud & Deployment

Supabase (PostgreSQL)
FastAPI on Vercel
Plotly Dash on Render
GitHub Actions Automation

MSc Data Analytics Graduate | Dublin, Ireland | June 2026

Dashboard: `https://financial-pipeline.onrender.com`
API: `https://financial-pipeline-seven.vercel.app`

Table of Contents

Chapter 1 — Project Introduction

1.1 What Is This Project?

This project is a **Financial Market Data Pipeline** — a complete, end-to-end data engineering system that automatically fetches stock market data, processes it through multiple layers of transformation, calculates financial indicators, and displays the results on a live dashboard accessible to anyone on the internet.

Think of it like a factory for data. Raw materials (stock prices) come in through one door. Workers (Spark, Python scripts) clean and transform those materials. The finished product (charts, indicators, insights) goes out the other door and onto a website for the world to see.

> **Live Project**
> This project is deployed live at `financial-pipeline.onrender.com` with data for 30 major US stocks including Apple, Microsoft, Google, Tesla, and NVIDIA — covering 2 full years of daily trading data with automated daily updates via GitHub Actions.

1.2 Why Build This?

This is the second portfolio project in an MSc Data Analytics portfolio. The purpose is to learn and demonstrate three tools that appear in job descriptions at companies like Revolut, Stripe, Zalando, Databricks, Google, and Amazon:

- **Docker** — the industry standard for packaging and running applications in isolated, reproducible environments
- **Apache Airflow** — the industry standard for scheduling, orchestrating, and monitoring data pipelines
- **Apache Spark / PySpark** — the industry standard for processing large datasets at scale

1.3 Technology Stack

Technology	Purpose	Why Chosen
Docker	Run all services in isolated Linux containers	Airflow and Spark don't work natively on Windows
Apache Airflow 2.9	Schedule and monitor the data pipeline	Industry standard orchestration tool
Apache Spark 3.5 / PySpark	Distributed data processing	Handles large scale data, industry standard
yfinance	Free stock price data	No API key required, completely free
Parquet files	Columnar storage for data lake	10x smaller than CSV, faster queries

Supabase	Cloud PostgreSQL database	Free, always accessible from anywhere
FastAPI	REST API serving Gold data	Fast, modern, auto-generates docs
Plotly Dash	Interactive charts dashboard	Python-native, works with DataFrames
GitHub Actions	Automated daily updates	Free cloud automation, no server needed
Vercel	FastAPI hosting	Free, fast, never sleeps
Render	Dashboard hosting	Free, persistent server, perfect for Dash

Chapter 2 — Core Concepts Explained Simply

2.1 What Is Docker?

Imagine you build a recipe on your kitchen at home. It works perfectly. You send the recipe to your friend but they have different spices, different equipment, a different oven. The dish tastes completely different. This is exactly the problem software developers face every day.

Docker solves this by putting your entire application — your code, your Python version, your libraries, your settings — inside a sealed box called a **container**. This box runs identically on any machine that has Docker installed.

> **Simple Analogy**
> Docker is like a shipping container. Before containers existed, loading a ship was chaotic. Shipping containers standardised everything — one box fits on any ship, any truck, any train. Docker does the same for software.

Docker Term	Plain English Explanation
Image	The recipe — a read-only template describing what should be in a container
Container	The running cake made from the recipe — a live instance of an image
Dockerfile	The recipe card — where you write instructions for building an image
Docker Compose	The conductor — starts and connects multiple containers together

2.2 What Is Apache Airflow?

Every data pipeline has the same problem: multiple steps need to run in a specific order, on a schedule, and if one step fails you need to prevent downstream steps from running on bad data.

> **Simple Analogy**
> Airflow is the head chef in a restaurant kitchen. The head chef doesn't cook — individual chefs do. The head chef coordinates: tells the starter chef to begin, tells the main course chef to start when starters are done, notices when something burns. Airflow is that coordinator for your data pipeline.

What Is a DAG?

DAG stands for **Directed Acyclic Graph**. It is a map of your tasks and their dependencies. In this project:

```
fetch_stock_data → bronze_to_silver → silver_to_gold → notify_complete
```

```
Each arrow means: "only run the next task if this one succeeded"
If fetch_stock_data fails → nothing else runs → no bad data downstream
```

2.3 What Is Apache Spark?

Pandas loads your entire dataset into RAM at once. If your dataset is bigger than your RAM, pandas crashes. Spark was built to solve this — it splits data across many machines and processes chunks in parallel.

> **Simple Analogy**
> Pandas is one person reading a book. Spark is 50 people each reading a different chapter simultaneously, then combining their summaries. The same work gets done far faster in parallel.

Use Pandas When...

- Data fits in RAM
- Quick exploratory analysis
- Daily updates (30 rows)
- Simple scripts

Use PySpark When...

- Data larger than RAM
- Production pipelines
- Historical bulk loads
- Processing millions of rows

2.4 The Medallion Architecture

A pattern for organising data in layers, each adding more quality and value. Popularised by Databricks (the company that built Spark).

BRONZE → **Raw data** — exactly as received from Yahoo Finance. Nothing changed. Your safety net.

SILVER → **Clean data** — nulls removed, types fixed, duplicates eliminated. Trusted data.

GOLD → **Business-ready data** — Moving Averages, RSI, Volatility calculated. Ready for dashboards.

Chapter 3 — System Architecture

3.1 The Two Environments

Local (Docker on Windows)

- Apache Airflow (scheduler + UI)
- Apache Spark (data processor)
- PostgreSQL (Airflow metadata)
- FastAPI (local testing)
- Plotly Dash (local testing)

Cloud (Always On)

- Supabase (PostgreSQL database)
- FastAPI on Vercel
- Dash on Render
- GitHub Actions (automation)
- GitHub (code repository)

3.2 Data Flow

```
DAILY AUTOMATED FLOW (6am weekdays):

GitHub Actions (cloud)
  └─▶ daily_update.py runs on GitHub's servers
      └─▶ yfinance fetches 30 stocks (fresh IP — no rate limiting)
          └─▶ pandas calculates indicators
              └─▶ writes to Supabase

DASHBOARD ACCESS (any time, anyone):

User opens browser
  └─▶ Dash on Render receives request
      └─▶ Dash calls FastAPI on Vercel
          └─▶ FastAPI queries Supabase
              └─▶ Returns JSON data
                  └─▶ Dash renders charts
```

3.3 Project Folder Structure

```
financial-pipeline/
├── .env                  ← Secret credentials (NEVER push to GitHub)
├── .gitignore            ← Tells Git what NOT to upload
├── docker-compose.yml    ← Defines and connects all Docker containers
├── daily_update.py       ← GitHub Actions automation script
```

```
├── load_to_supabase.py      ← One-time data load script
│
├── airflow/
│   ├── Dockerfile           ← Custom Airflow image recipe
│   └── dags/
│       └── market_pipeline.py   ← The DAG definition
│
├── spark/
│   └── jobs/
│       ├── bronze_to_silver.py  ← PySpark cleaning
│       └── silver_to_gold.py    ← PySpark indicators
│
├── ingestion/
│   └── fetch_stocks.py          ← yfinance data fetcher
│
├── api/
│   ├── main.py                  ← FastAPI application
│   └── vercel.json              ← Vercel deployment config
│
├── dashboard/
│   └── app.py                   ← Plotly Dash application
│
├── data/                        ← Local data lake (gitignored)
│   ├── bronze/  ├── silver/  └── gold/
│
└── .github/workflows/
    └── daily_update.yml         ← Automated schedule
```

Chapter 4 — Environment Setup

4.1 Installing Docker on Windows 11

Step 1 — Enable WSL2

```
# Open PowerShell as Administrator:
wsl --install
# Restart when prompted, then:
wsl --set-default-version 2
```

Step 2 — Install Docker Desktop

Download from `https://www.docker.com/products/docker-desktop/` and run the installer. During installation ensure "Use WSL2 instead of Hyper-V" is checked.

Step 3 — Verify Installation

```
docker --version
# Expected: Docker version 26.x.x

docker compose version
# Expected: Docker Compose version v2.x.x

docker run hello-world
# Expected: Hello from Docker!
```

4.2 Creating Project Structure

```
cd C:\Users\YourName\Desktop
mkdir financial-pipeline
cd financial-pipeline

mkdir airflow, airflow/dags, airflow/logs, airflow/plugins
mkdir spark, spark/jobs
mkdir ingestion, api, dashboard
mkdir data, data/bronze, data/silver, data/gold

code .    # Opens VS Code
```

Chapter 5 — Docker Configuration

5.1 The Airflow Dockerfile

Takes the official Airflow image and adds Java (required by Spark) plus our Python libraries.

```
FROM apache/airflow:2.9.3

USER root    # Switch to admin to install system software

RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        openjdk-17-jre-headless \    # Java — required by PySpark
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*

ENV JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64  # Tell Spark where Java lives

USER airflow    # Switch back to limited user

RUN pip install --no-cache-dir \
    yfinance \
    pandas==2.2.2 \
    pyarrow==16.0.0 \
    pyspark==3.5.0
```

5.2 Docker Compose — The Conductor

Defines all six services and how they connect. Key concepts:

- **x-airflow-common** — YAML anchor. Defines shared settings once, reused by all three Airflow services.
- **depends_on with service_healthy** — Airflow waits for PostgreSQL to fully start before launching.
- **volumes** — Maps local folders into containers so code changes are reflected instantly.
- **ports** — Maps container ports to host ports (8080:8080 means your browser hits port 8080 on localhost, which Docker forwards to the container).

```
services:
  postgres:          # Airflow's metadata database
```

```
  image: postgres:15
  healthcheck:
    test: ['CMD', 'pg_isready', '-U', 'airflow']

airflow-webserver: # The visual UI at localhost:8080
  build: ./airflow
  command: webserver
  ports: ['8080:8080']

airflow-scheduler: # Watches the clock, triggers tasks
  build: ./airflow
  command: scheduler

spark:             # Data processing engine
  image: apache/spark:3.5.0
  ports: ['8081:4040']

api:               # FastAPI serving Gold data
  build: ./api
  environment:
    - SUPABASE_URL=${SUPABASE_URL}    # Read from .env file
    - SUPABASE_KEY=${SUPABASE_KEY}
  ports: ['8000:8000']

dashboard:         # Plotly Dash
  build: ./dashboard
  ports: ['8050:8050']
```

Chapter 6 — Data Ingestion (Bronze Layer)

6.1 The fetch_stocks.py Script

Loops through all 30 stocks, fetches 2 years of daily data, saves as Parquet.

```
import yfinance as yf
import pandas as pd
from datetime import datetime, timedelta
import os

STOCKS = ['AAPL', 'MSFT', 'GOOGL', 'AMZN', 'META', ...]
BRONZE_PATH = '/opt/airflow/data/bronze'

def fetch_stock_data(symbol, start_date, end_date):
    ticker = yf.Ticker(symbol)      # Create object representing this stock
    df = ticker.history(start=start_date, end=end_date)  # Fetch data
    df.reset_index(inplace=True)    # Make Date a regular column, not index
    df.columns = [col.lower().replace(' ', '_') for col in df.columns]
    df['date'] = pd.to_datetime(df['date']).dt.tz_localize(None)
    df['symbol'] = symbol           # Tag each row with the stock symbol
    return df

def save_to_bronze(df, symbol):
    os.makedirs(BRONZE_PATH, exist_ok=True)
    file_path = os.path.join(BRONZE_PATH, f'{symbol}.parquet')
    df.to_parquet(
        file_path,
        index=False,
        coerce_timestamps='us',          # Critical: microseconds for Spark
        allow_truncated_timestamps=True   # Critical: prevent timestamp errors
    )

def run_ingestion():
    end_date = datetime.now().strftime('%Y-%m-%d')
    start_date = (datetime.now() - timedelta(days=730)).strftime('%Y-%m-%d')

    for symbol in STOCKS:
        try:
            df = fetch_stock_data(symbol, start_date, end_date)
            if len(df) == 0:    # Critical: never save empty DataFrames
```

```
            continue
        save_to_bronze(df, symbol)
    except Exception as e:
        print(f'Failed {symbol}: {e}')  # Log but keep going
```

6.2 Why Parquet?

Feature	CSV	Parquet
Storage	Stores all columns every row	Columnar — read only what you need
File size	Large — no compression	10x smaller — built-in compression
Type safety	Everything is text	Types stored explicitly
Query speed	Must read entire file	Skip unneeded columns
Industry use	Good for sharing data	Standard for data lakes

Chapter 7 — PySpark Data Processing

7.1 Bronze to Silver — Cleaning

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

def create_spark_session():
    return SparkSession.builder \
        .appName('BronzeToSilver') \
        .master('local[*]') \       # Use ALL available CPU cores
        .getOrCreate()

def clean_data(df):
    df = df.filter(F.col('close').isNotNull())    # Remove missing prices
    df = df.filter(F.col('close') > 0)            # Remove impossible prices
    df = df.dropDuplicates(['date', 'symbol'])    # Remove duplicates
    df = df.withColumn('close', F.col('close').cast(DoubleType()))  # Fix types
    return df

def write_silver_data(df):
    df.coalesce(1).write \    # coalesce(1) = write as single file
        .mode('overwrite') \
        .parquet(SILVER_PATH)
```

7.2 Silver to Gold — Technical Indicators

Window Functions — The Core Concept

A Window function calculates a value for each row based on a sliding window of surrounding rows. Think of it as a magnifying glass that slides along your data, calculating something for whatever it can see through the glass.

```
from pyspark.sql import Window

# Define the window: for each stock, ordered by date
window = Window.partitionBy('symbol').orderBy('date')

# 7-day window: current row and 6 before it
window_7 = window.rowsBetween(-6, 0)
```

```
# Moving average: average of closes within the window
df = df.withColumn('ma_7', F.avg(F.col('close')).over(window_7))
df = df.withColumn('ma_30', F.avg(F.col('close')).over(window_30))

# Daily return: percentage change from yesterday
df = df.withColumn('daily_return',
    (F.col('close') - F.lag('close', 1).over(window)) /
     F.lag('close', 1).over(window) * 100
)

# Volatility: standard deviation of returns
df = df.withColumn('volatility_30',
    F.stddev(F.col('daily_return')).over(window_30)
)

# RSI calculation (simplified):
df = df.withColumn('gain', F.when(F.col('daily_return') > 0, F.col('daily_return')).otherwise(0))
df = df.withColumn('loss', F.when(F.col('daily_return') < 0, F.abs(F.col('daily_return'))).otherwise(0))
df = df.withColumn('avg_gain', F.avg('gain').over(window_14))
df = df.withColumn('avg_loss', F.avg('loss').over(window_14))
df = df.withColumn('rsi', 100 - (100 / (1 + F.col('avg_gain') / F.col('avg_loss'))))
```

Chapter 8 — Airflow DAG

8.1 The Complete DAG

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.operators.bash import BashOperator

default_args = {
    'owner': 'data_engineer',
    'retries': 2,                        # Retry twice on failure
    'retry_delay': timedelta(minutes=5),  # Wait 5 minutes between retries
}

with DAG(
    dag_id='market_data_pipeline',
    default_args=default_args,
    schedule_interval='0 6 * * 1-5',  # 6am every Mon-Fri
    start_date=datetime(2024, 1, 1),
    catchup=False,                    # Don't backfill missed runs
) as dag:

    def run_ingestion_task():
        import sys
        sys.path.insert(0, '/opt/airflow')
        from ingestion.fetch_stocks import run_ingestion
        run_ingestion()

    fetch_data = PythonOperator(
        task_id='fetch_stock_data',
        python_callable=run_ingestion_task,
    )

    bronze_to_silver = BashOperator(
        task_id='bronze_to_silver',
        bash_command='spark-submit /opt/spark/jobs/bronze_to_silver.py',
    )

    silver_to_gold = BashOperator(
        task_id='silver_to_gold',
        bash_command='spark-submit /opt/spark/jobs/silver_to_gold.py',
```

```
)

notify_complete = PythonOperator(
    task_id='notify_complete',
    python_callable=lambda: logger.info('Pipeline complete'),
)

# The dependency chain — >> means "then run"
fetch_data >> bronze_to_silver >> silver_to_gold >> notify_complete
```

8.2 Understanding the Cron Schedule

Field	Value	Meaning
Minute	0	At the start of the hour (minute 0)
Hour	6	At 6am
Day of Month	*	Every day of the month
Month	*	Every month
Day of Week	1-5	Monday(1) through Friday(5) only

Combined: `0 6 * * 1-5` = Run at exactly 6:00am every Monday through Friday

Chapter 9 — FastAPI and Dashboard

9.1 FastAPI Endpoints

Endpoint	Returns
`GET /health`	API status check
`GET /api/stocks`	List of all 30 stock symbols
`GET /api/stocks/{symbol}`	Full historical data for one stock
`GET /api/stocks/{symbol}/latest`	Most recent row only
`GET /api/summary`	Latest data for all 30 stocks

Example Response

```
GET https://financial-pipeline-seven.vercel.app/api/stocks/AAPL

{
  "symbol": "AAPL",
  "count": 501,
  "data": [
    {
      "date": "2024-06-03",
      "open": 194.03,
      "close": 194.35,
      "ma_7": 189.45,
      "ma_30": 186.72,
      "rsi": 62.4,
      "volatility_30": 0.95,
      "daily_return": 0.17
    },
    // ... 500 more rows
  ]
}
```

9.2 Plotly Dash Dashboard Features

- **Stock dropdown** — select any of the 30 stocks
- **Time period filter** — 1 month, 3 months, 6 months, 1 year, 2 years
- **Price chart** — Close price with MA7 and MA30 overlaid

- **RSI chart** — With overbought (70) and oversold (30) reference lines
- **Volatility chart** — 30-day rolling standard deviation
- **Market Summary table** — Latest data for all 30 stocks at a glance
- **Auto-refresh** — Updates every 60 seconds

Chapter 10 — Deployment

10.1 Supabase Setup

```
-- Create table in Supabase SQL Editor:
CREATE TABLE gold_market_data (
    id            BIGSERIAL PRIMARY KEY,
    symbol        VARCHAR(10) NOT NULL,
    date          DATE NOT NULL,
    open          DOUBLE PRECISION,
    close         DOUBLE PRECISION,
    ma_7          DOUBLE PRECISION,
    ma_30         DOUBLE PRECISION,
    daily_return  DOUBLE PRECISION,
    volatility_30 DOUBLE PRECISION,
    rsi           DOUBLE PRECISION,
    UNIQUE(symbol, date)
);

-- Create functions for efficient queries:
CREATE OR REPLACE FUNCTION get_distinct_symbols()
RETURNS TABLE(symbol VARCHAR) AS $$
BEGIN
    RETURN QUERY SELECT DISTINCT gmd.symbol
    FROM gold_market_data gmd ORDER BY gmd.symbol;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE FUNCTION get_latest_per_symbol()
RETURNS TABLE(symbol VARCHAR, date DATE, close FLOAT8, ...) AS $$
BEGIN
    RETURN QUERY
    SELECT DISTINCT ON (gmd.symbol)
        gmd.symbol, gmd.date, gmd.close, ...
    FROM gold_market_data gmd
    ORDER BY gmd.symbol, gmd.date DESC;
END;
$$ LANGUAGE plpgsql;
```

10.2 GitHub Actions Automation

```
# .github/workflows/daily_update.yml
name: Daily Stock Data Update

on:
  schedule:
    - cron: '0 6 * * 1-5'   # 6am every weekday
  workflow_dispatch:          # Manual trigger from GitHub UI

jobs:
  update-data:
    runs-on: ubuntu-latest    # Fresh Linux server, fresh IP address

    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: pip install yfinance pandas numpy supabase

      - name: Run daily update
        env:
          SUPABASE_URL: ${{ secrets.SUPABASE_URL }}   # Stored in GitHub Secrets
          SUPABASE_KEY: ${{ secrets.SUPABASE_KEY }}
          TABLE_NAME: ${{ secrets.TABLE_NAME }}
        run: python daily_update.py
```

10.3 Live URLs

Service	Platform	URL
Dashboard	Render	https://financial-pipeline.onrender.com
API	Vercel	https://financial-pipeline-seven.vercel.app
API Health	Vercel	https://financial-pipeline-seven.vercel.app/health
GitHub	GitHub	https://github.com/Ob09/financial-pipeline

Chapter 11 — Challenges and Solutions

Challenge 1 — Yahoo Finance Rate Limiting

Problem: Docker container's IP was blocked after hundreds of debug requests. yfinance returned empty DataFrames.

Solution: Downloaded data via local Anaconda as a workaround. Added empty DataFrame detection in fetch_stocks.py. GitHub Actions uses fresh IPs and makes only 30 requests/day.

Lesson: Always handle empty API responses explicitly. Never save empty data silently.

Challenge 2 — Parquet Timestamp Incompatibility

Problem: Local pyarrow used nanosecond timestamps; Spark 3.5 requires microseconds. Error: `Illegal Parquet type: INT64 (TIMESTAMP(NANOS,true))`

Solution: Added `coerce_timestamps='us'` and `allow_truncated_timestamps=True` to all Parquet writes. Converted date columns to `datetime64[us]`.

Challenge 3 — Supabase 1000-Row Limit

Problem: Supabase returns max 1000 rows per request. Dataset has 15,030 rows. Queries returned only 2 stocks.

Solution: Created PostgreSQL functions that execute in the database and return only the required aggregated data. No row limit for server-side functions.

Challenge 4 — Dash on Vercel

Problem: Dash crashed on Vercel with FUNCTION_INVOCATION_FAILED. Vercel is serverless — it cannot maintain Dash's persistent server process.

Solution: FastAPI on Vercel (stateless, perfect for APIs), Dash on Render (persistent, perfect for Dash). Right tool for the right job.

Chapter 12 — Key Learnings

12.1 Docker

- Docker is not just for deployment — it solves the "works on my machine" problem during development
- WSL2 gives Windows machines a genuine Linux kernel, enabling Linux-only tools like Airflow
- Volume mounts allow local code changes to be reflected inside containers instantly
- Docker Compose makes multi-service architectures manageable with a single command

12.2 Apache Airflow

- A DAG is a dependency map — it defines what runs after what, not just a schedule
- Separating webserver and scheduler follows single responsibility principle
- Task failures cascade correctly — Bronze failure prevents Silver running on bad data
- The Airflow UI provides immediate visibility into pipeline health

12.3 Apache Spark

- Lazy evaluation means Spark plans before executing — enabling powerful optimization
- Window functions are the core pattern for time-series calculations
- Spark for bulk historical loads; pandas for small daily incremental updates
- coalesce(1) writes a single output file — simpler for subsequent reading

12.4 Architecture Decisions

- Medallion Architecture separates raw, clean, and enriched data — each layer serves different audiences
- Supabase bridges local processing and cloud deployment elegantly and for free
- Right platform for each service matters more than one platform for everything
- GitHub Actions provides powerful free cloud automation for scheduled tasks

Chapter 13 — How to Run the Project

13.1 Start Locally

```
# 1. Clone repository
git clone https://github.com/Ob09/financial-pipeline.git
cd financial-pipeline

# 2. Create .env file (see Chapter 5 for content)

# 3. Start all Docker containers
docker compose up --build

# Wait for: "Listening at: http://0.0.0.0:8080"

# 4. Access services:
# Airflow UI:  http://localhost:8080  (admin/admin)
# Dashboard:   http://localhost:8050
# API:         http://localhost:8000/health
# Spark UI:    http://localhost:8081

# 5. Stop everything
docker compose down
```

13.2 Load Data

```
# Download data (Anaconda Prompt):
python download_data.py

# Run Spark pipeline:
docker exec financial-pipeline-airflow-scheduler-1 \
  spark-submit /opt/spark/jobs/bronze_to_silver.py

docker exec financial-pipeline-airflow-scheduler-1 \
  spark-submit /opt/spark/jobs/silver_to_gold.py

# Load to Supabase:
pip install supabase python-dotenv
python load_to_supabase.py
```

13.3 Trigger GitHub Actions

- Go to GitHub repository → Actions tab
- Click "Daily Stock Data Update" in sidebar
- Click "Run workflow" → "Run workflow" (green button)
- Should complete in under 60 seconds with all green checkmarks

Appendix — Technical Indicators Reference

Moving Average

```
MA(n) = Sum of closing prices over last n days / n

Example — 7-day MA:
Prices: 150, 152, 149, 153, 155, 151, 154
MA7 = (150+152+149+153+155+151+154) / 7 = 152.0

Signal interpretation:
  MA7 > MA30  → Short-term stronger than long-term → Bullish trend
  MA7 < MA30  → Short-term weaker than long-term  → Bearish trend
  MA7 crosses above MA30 → Golden Cross → Buy signal
  MA7 crosses below MA30 → Death Cross  → Sell signal
```

RSI — Relative Strength Index

```
Step 1: avg_gain = average of up-day returns over 14 days
Step 2: avg_loss = average of down-day returns over 14 days
Step 3: RS = avg_gain / avg_loss
Step 4: RSI = 100 - (100 / (1 + RS))

Scale:
  RSI > 70  → Overbought — price rose too fast, may drop
  RSI < 30  → Oversold  — price fell too fast, may rise
  RSI ≈ 50  → Neutral momentum
```

Volatility

```
Volatility = Standard Deviation of Daily Returns over 30 days

Daily Return = (Today Close - Yesterday Close) / Yesterday Close × 100

High volatility = large swings, higher risk, higher potential reward
Low volatility  = stable movement, lower risk, lower potential reward
```

End of Report — Financial Market Data Pipeline
MSc Data Analytics | Dublin, Ireland | June 2026